

Security Audit

Lizcoin (Token)

Table of Contents

Executive Summary	4
Project Context	4
Audit scope	7
Security Rating	8
Intended Smart Contract Functions	9
Code Quality	10
Audit Resources	10
Dependencies	10
Severity Definitions	11
Status Definitions	12
Audit Findings	13
Centralisation	18
Conclusion	19
Our Methodology	20
Disclaimers	22
About Hashlock	23

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The Lizard Labs team partnered with Hashlock to conduct a security audit of their BedrockERC20.sol smart contract. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

Lizard Labs specializes in Web3 innovation, focusing on decentralized technologies and community-driven projects. They provide tools for streamlined project development and blockchain integration. Their services and solutions cater to businesses seeking to leverage blockchain for transparency, security, and user empowerment. To build engagement, they emphasize community-focused strategies, reward mechanisms, and scalable blockchain solutions. By merging Web3 tech with marketing strategies like token-based loyalty programs and NFT-based brand experiences, Lizard Labs enables brands to foster deeper connections with digital-native audiences

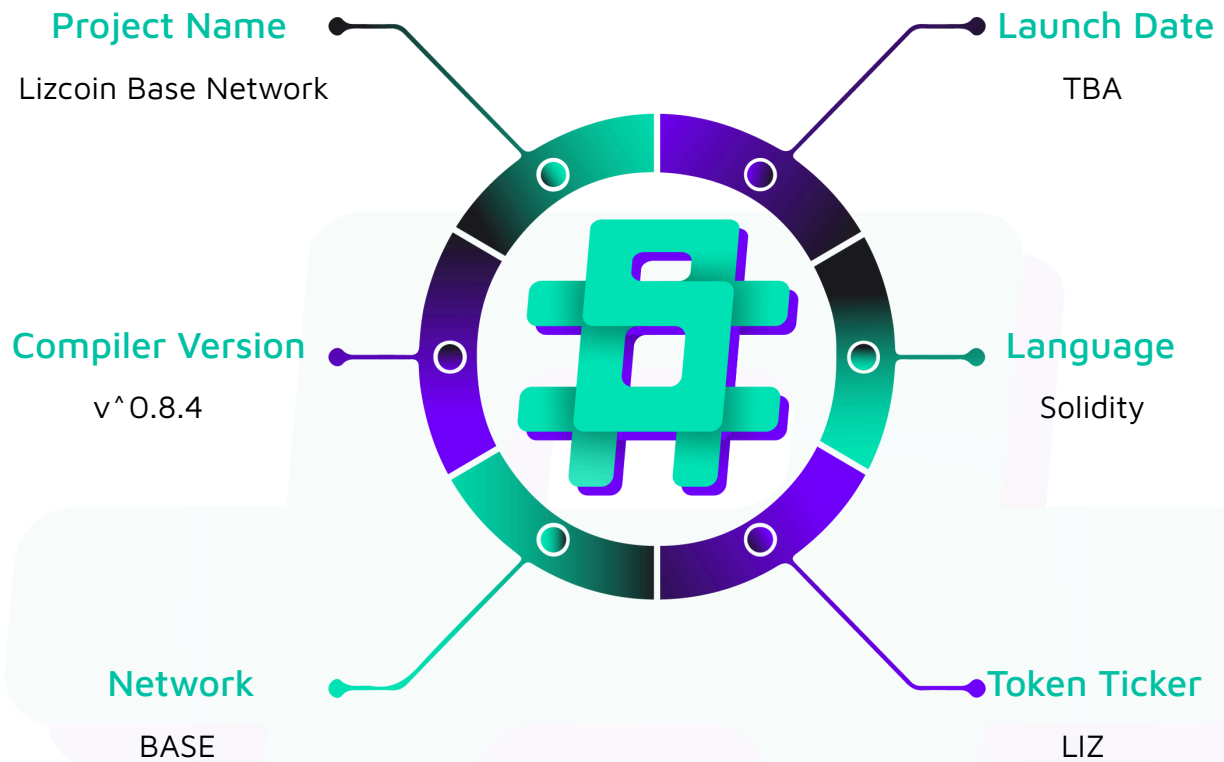
Project Name: Lizcoin Base Network

Compiler Version: ^0.8.4

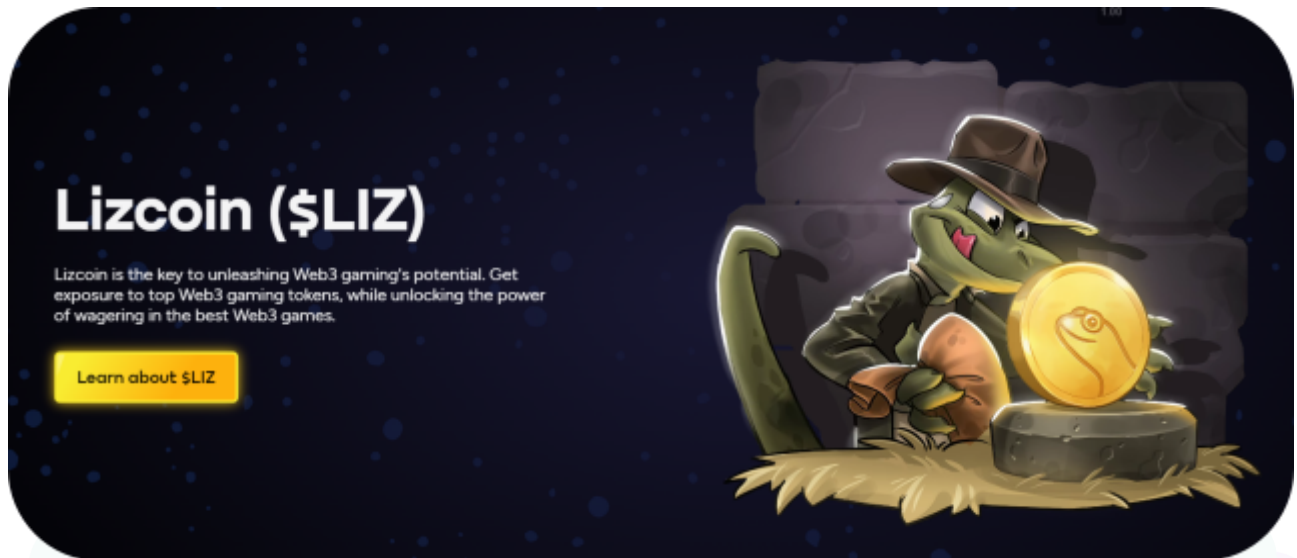
Website: www.lizlabs.io/ethlizards

Logo:



Visualised Context:

Project Visuals:



Audit scope

We at Hashlock audited the solidity code within the Lizcoin Base Network project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	LizardLabs Protocol Smart Contracts
Platform	Base / Solidity
Audit Date	December, 2024
Contract 1	BedrockERC20.sol
Contract 1 MD5 Hash	2ac12943bcfd925b9d3440e79d945f30
GitHub Commit Hash	29f329b612abf9c5b82463159734e2da5bb2341d

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering.



Not Secure

Vulnerable

Secure

Hashlocked

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The general security overview is presented in the [Standardised Checks](#) section and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

All vulnerabilities initially identified have now been acknowledged.

Hashlock found:

1 Medium severity vulnerabilities

1 Low severity vulnerabilities

1 Gas Optimisations

2 QAs

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
BedrockERC20.sol - Bedrock compatible version of the Advanced ERC20 - Implements IOptimismMintableERC20, AdvanceERC20	Contract achieves this functionality.

Code Quality

This audit scope involves the smart contracts of the Lizcoin Base Network project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring is recommended in the future.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Lizcoin Base Network project smart contract code in the form of Github access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

Medium

[M-01] BedrockERC20 - Bridging tokens can allow users to double vote

Description

The BedrockERC20 token inherits the AdvancedERC20 contract that has voting mechanisms implemented in it. Bridging tokens can potentially lead to users utilizing their voting power twice on the L1 and L2 chains without acquiring new tokens.

Vulnerability Details

The `__mint` function is called by the bridge when users bridge to L2

```
function __mint(address to, uint256 value) private {
    // non-zero recipient address check
    require(to != address(0), "zero address");

    // non-zero value and arithmetic overflow check on the total supply
    // this check automatically secures arithmetic overflow on the individual balance
    unchecked {
        // put operation into unchecked block to display user-friendly overflow
        error message for Solidity 0.8+
        require(totalSupply + value > totalSupply, "zero value or arithmetic
overflow");
    }

    // uint192 overflow check (required by voting delegation)
    require(totalSupply + value <= type(uint192).max, "total supply overflow
(uint192)");

    // perform mint:
    // increase total amount of tokens value
    totalSupply += value;
```

```

// increase `to` address balance
tokenBalances[to] += value;

// update total token supply history
__updateHistory(totalSupplyHistory, add, value);

// create voting power associated with the tokens minted
__moveVotingPower(msg.sender, address(0), votingDelegates[to], value);

// fire a minted event
emit Minted(msg.sender, to, value);

// emit an improved transfer event (arXiv:1907.00903)
emit Transfer(msg.sender, address(0), to, value);

// fire ERC20 compliant transfer event
emit Transfer(address(0), to, value);
}

```

As observed in the highlighted part, users will receive voting power according to the amount that was minted.

Think of the scenario where the user has 1000 tokens and for simplicity assume 1 token = 1 voting power.

- User has 1000 tokens on L1 which is 1000 voting power.
- User, uses this voting power to vote on a certain DAO decision
- User then bridges their tokens to L2
- User gets minted 1000 tokens on L2 which is once again 1000 voting power
- User, uses this voting power to vote on a certain DAO decision

Depending on the voting mechanisms, this could allow users to double vote with their existing tokens.

Impact

Users can double vote with their existing tokens.

Recommendation

Voting mechanism should use a snapshot mechanic that will count a user's both L1 and L2 tokens at that time when they try to vote.

Note

The Lizardlabs team informed Hashlock that will resolve all the findings at a future date.

Status

Acknowledged

Low

[L-01] BedrockERC20 - Centralization risk for privileged roles

Description

The project uses the `InitializableAccessControlCore.sol` contract from `lazy-sol` to implement access control on critical functionality such as token minting and burning. Minting and burning permissions are necessary for the bridge contract. However, the owner of the contract has unlimited access to all functions such as minting any amount of tokens or burning any amount of tokens from any user. This can introduce problems such as the token amount going over the intended total supply and bricking bridge functionality, it also introduces a single point of failure.

Recommendation

To mitigate centralization risks associated with privileged functions, consider implementing a multi-signature or decentralized governance mechanism. Instead of relying solely on a single owner/administrator, distribute control and decision-making



authority among multiple parties or stakeholders. This approach enhances security, reduces the risk of malicious actions by a single entity, and prevents single points of failure.

Status

Acknowledged

Gas

[G-01] BedrockERC20#__bridgeSetup - Consider using custom errors for gas efficiency

Description

Custom errors provide a way for developers to define more descriptive and semantically meaningful error conditions without relying on string messages. Every unique string used as a revert reason consumes gas, making transactions more expensive.

Recommendation

Utilize custom errors. An example is shown below.

```
function __bridgeSetup(address remoteTokenAddr) private {  
    // verify the inputs are set  
    if(remoteTokenAddr == address(0)) revert ZeroAddress();  
    // set contract's internal state  
    REMOTE_TOKEN = remoteTokenAddr;  
    // bridge is assumed to have mint/burn permissions by default  
    updateRole(BRIDGE, ROLE_TOKEN_CREATOR | ROLE_TOKEN_DESTROYER);  
}
```

Status

Acknowledged

QA

[Q-01] BedrockERC20 - Avoid using floating pragma unless necessary

Description

A "floating pragma" in Solidity refers to the practice of using a pragma statement that does not specify a fixed compiler version but instead allows the contract to be compiled with any compatible compiler version. This can lead to various compatibility and stability issues.

Recommendation

Consider locking the pragma version whenever possible and avoid using a floating pragma in the final deployment. Consider [known bugs](#) for the compiler version that is chosen.

Status

Acknowledged

[Q-02] BedrockERC20 - Function ordering does not follow the Solidity style guide

Description

According to the Solidity [style guide](#), functions should be laid out in the following order: constructor(), receive(), fallback(), external, public, internal, private, but the contract does not follow this pattern.

Recommendation

Follow the official Solidity guidelines.

Status

Acknowledged

Centralisation

The Lizcoin - Base Network project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.

LizardLabs emphasizes a security-focused approach while ensuring operational functionality. By centralizing critical functions, the project enhances reliability and rapid decision-making while maintaining accountability, ensuring the system remains robust and efficient.



Centralised

Decentralised

Conclusion

After Hashlock's analysis, the Lizcoin Base Network project seems to have a sound and well-tested code base, however, our findings have been acknowledged and we suggest to be resolved to achieve full security. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.